



---

# Transportation's Systems 20-5751

---

## PROJECT PART 1: ALL-OR-NOTHING ASSIGNMENT

PROF. ZAHRA AMINI  
KEIVAN JAMALI

1405/01/23

# 1 Introduction to Traffic Assignment

Traffic assignment is a fundamental process in transportation engineering that aims to predict the flow of traffic across a network given a set of origins, destinations, and travel demands. The "All-or-Nothing" (AON) assignment is the simplest form of traffic assignment. In an AON assignment, all traffic demand between an origin-destination (O-D) pair is assigned to the **shortest path** based on a fixed travel time for each link. This method is a crucial first step as it forms the basis for more complex assignment algorithms.

The core idea is:

1. Define travel times for all links.
2. For each O-D pair, find the shortest path using these travel times.
3. Assign the entire demand for that O-D pair to the links that constitute its shortest path.

## 2 Generating Link Travel Time Functions

For a realistic simulation, link travel times should not be static but rather depend on the flow or other characteristics. However, for the AON assignment, we will use fixed travel times for simplicity. These fixed travel times will be generated algorithmically using student IDs to ensure unique inputs for each student.

Each student will use their unique `student_id` to generate parameters for link travel time functions.

**Travel Time Generation Algorithm:** For each link  $(i, j)$  in the network with length  $L_{ij}$ :

1. **Deterministic Seed Generation:** Use the `student_id` as a seed for Python's built-in `hashlib.sha3_256()` function (first import `hashlib`). This ensures a deterministic output for each student ID.

- $\text{seed} = \text{int}(\text{hashlib.sha3\_256}(\text{bytes}(\text{str}(\text{id}), \text{"utf-8"})).\text{hexdigest}(), 16)$

2. **Parameter Generation:** Based on the `seed_value`, generate two parameters,  $A$  and  $B$ , using modulo arithmetic for consistency:

- Let  $A = (\text{seed} \bmod 100) / 100.0 + 1.0$
- Let  $B = (\text{seed} \bmod 50) / 100.0 + 0.5$

These values will ensure variability across students based on their unique ID.

3. **Base Travel Time:** Calculate a base travel time  $T_{ij}^0$  in minutes for link  $(i, j)$  using its length  $L_{ij}$  in km and the generated parameters:

$$T_{ij}^0 = A \times L_{ij}$$

4. **Adding Noise/Flow Dependency (for future parts, but parameterized here):** While AON uses fixed times, we can define a function that *would* produce variable times. This prepares for later parts. For Part 1, we use the **Base Travel Time** as the fixed travel time. For demonstration and for use in subsequent parts, a simplified BPR-like function can be considered:

$$T_{ij}(f_{ij}) = T_{ij}^0 \left( 1 + B \left( \frac{f_{ij}}{C_{ij}} \right)^4 \right)$$

where  $f_{ij}$  is the flow on link  $(i, j)$  and  $C_{ij}$  is the link capacity. For Part 1, we **assume a default capacity**  $C_{ij} = 1000$  for all links and **use**  $f_{ij} = 0$  to get the free-flow travel time:

$$\text{Fixed Travel Time}_{ij} = T_{ij}^0 = A \times L_{ij}$$

**Note on SymPy:** While SymPy can be used to manipulate these functions symbolically, for Part 1, we only need to calculate the *fixed travel time* derived from  $T_{ij}^0$ . Students should implement the generation of  $A$  and  $B$  and then compute  $T_{ij}^0$  for each link.

### 3 Defining Travel Demand

For the Sioux Falls network, we will generate a simplified demand matrix using student IDs to ensure uniqueness.

**Demand Generation Algorithm:** For each node pair  $(o, d)$  where  $o \neq d$ :

1. **Deterministic Seed Generation:** Use the `student_id` along with the specific origin and destination nodes to create a unique seed for each O-D pair.

- `od_seed_value = int(hashlib.sha3_256(  
bytes(str(student_id) + str(o) + str(d), "utf-8")  
)hexdigest(), 16)`

2. **Demand Value:** Generate the demand  $D_{od}$  from origin  $o$  to destination  $d$  as follows:

$$D_{od} = (\text{od\_seed\_value} \pmod{100}) + 10$$

This will produce a demand value between 10 and 109 (inclusive) for each O-D pair.

**Implementation Note:** The demand matrix should be represented using a suitable structure, such as a pandas DataFrame or a dictionary of dictionaries, mapping  $(o, d)$  pairs to their demand values.

## 1.4 Performing the All-or-Nothing Assignment

With the fixed link travel times and the O-D demand matrix generated, you can now perform the AON assignment.

**Steps:**

1. **Construct the Network Graph:** Use the graph constructed in PART 0. Ensure link lengths are loaded as attributes.
2. **Generate Travel Times:** Implement the algorithm in Section 1.2 to calculate a fixed travel time for each link using your `student_id` and Python's `hash()` function. Add these travel times as edge attributes (e.g., named `travel_time`) to your graph.
3. **Generate Demand:** Implement the algorithm in Section 1.3 to create the O-D demand matrix using your `student_id` and the specified hashing for each O-D pair.
4. **Find Shortest Paths:** For every O-D pair  $(o, d)$  with non-zero demand  $D_{od}$ :
  - Use NetworkX's shortest path algorithms (e.g., `nx.shortest_path` with `weight='travel_time'`) to find the shortest path from  $o$  to  $d$ .
  - Ensure you use the generated fixed travel times as weights.
5. **Assign Demand to Links:** For each shortest path found, iterate through its links. For each link  $(i, j)$  on the shortest path, add the demand  $D_{od}$  to a running total of flow for that link. Initialize all link flows to zero before starting this step. Store these final link flows.

## 4 Output Requirements

Students must submit the following:

1. **Python Script:** The complete, runnable Python script that performs all steps: network loading, travel time generation, demand generation, AON assignment, and calculation of the final checkable number.
2. **Small Report:** A brief report (e.g., a plain text file or a short markdown document) containing:
  - Your `student_id`.
  - The parameters  $A$  and  $B$  generated for three arbitrarily chosen links, showing they were derived using the specified `hash()` method.
  - Resultant network with the flows you calculated in this task.
3. **IMPORTANT\_Number:** A single numerical value that summarizes the AON assignment results.
  - sum over all the "flow" values on all links in the belowing way:  
 $\text{hash}((\text{start node as int, end node as int, flow on the link as float})) \bmod 10,000$
  - Report the **Number** as your final checkable number.

$$\text{Checkable Number} = \sum_{\text{all links } (i,j)} \text{hash}((\text{Node}_i, \text{Node}_j, \text{flow}_{ij})) \% 10000$$